

TrajectoryDB: A New Database for Agent Trajectories

<https://arxiv.org/abs/2609.07782>

1. Problem

- 문제: Agent trajectory가 JSONL, DB, observability/log 시스템 등에 분산되어 있어 전체 trajectory를 구조·순서·의미·provenance 관점에서 통합적으로 저장/조회하기 어렵다.
- 왜 중요한가: Agent가 길고 복잡해질수록 trajectory가 단순 로그가 아니라 memory, fine-tuning, debugging, safety, cost optimization의 핵심 데이터가 되기 때문이다.

2. Method

- 핵심 아이디어: Agent trajectory를 별도의 first-class data type으로 보고, 이를 위한 trajectory-native DB인 TrajectoryDB를 설계하자는 제안.
- 기존 방식과 차이:
 - 기존: log DB + vector DB + graph DB + 별도 Python pipeline 등을 조합
 - TrajectoryDB: ingestion + storage + query processing을 trajectory에 맞춰 co-design
- 가장 중요한 구성요소:
 - Structure-aware retrieval: session → turn → step → message 구조와 branch/subagent 관계를 직접 관리
 - Sequence-aware query: 단순 record 검색이 아니라 A → B → C 같은 실행 순서를 query 대상으로 취급
 - LLM as UDF: 자유 텍스트에 대한 semantic judgment를 DB query operator로 통합
 - Lineage: memory/judgment/failure 등이 어떤 trajectory의 어떤 evidence에서 생성됐는지 추적

- Incremental semantic evaluation: trajectory가 계속 추가될 때 기존 semantic judgment를 재평가/갱신
- Trajectory-aware storage/compression: 반복되는 context/prefix를 활용해 대규모 text 중복을 줄이는 방향

3. Experiments & Results

- 실험 성격: 완성된 DB 시스템의 benchmark 논문이 아니라 실제 production trace를 이용한 workload characterization + system design 제안.
- Harvard의 production system에서 1개월 trace를 분석.
- 주요 관찰:
 - 80%의 session에서 context mutation 발생 → 단순 prefix dedup만으로는 부족
 - 5,951개 session 분석에서 fork가 발생하는 session들이 전체 instance의 80%+를 차지
 - PostgreSQL export에서 Zstandard는 약 20× compression, PostgreSQL 기본 compression은 2× 미만
 - 극단적인 대규모 Agent 환경에서는 naive하게 trajectory를 저장할 경우 약 35 PB/day 수준의 데이터가 발생할 수 있다고 추정
- 무엇이 개선됐는가: 구현된 TrajectoryDB의 성능 향상을 측정한 결과는 아직 제시되지 않음. 대신 기존 DB가 해결하기 어려운 7가지 핵심 시스템 challenge를 도출함.

4. Insight

- 핵심 기여 한 줄: Agent trajectory는 로그가 아니라, 구조·순서·semantic·lineage를 동시에 가진 새로운 DB workload다.
- 한계 / 아쉬운 점:
 - 현재는 Vision paper라 실제 TrajectoryDB 구현 및 end-to-end benchmark가 없음.
 - 어떤 storage/query architecture가 최종적으로 가장 좋은지는 아직 open problem.
 - 따라서 현재 논문의 핵심 가치는 성능 개선 증명보다 문제 정의와 연구 agenda 제시에 있음.
- 내가 가져갈 포인트:

- Agent 시스템에서 앞으로 중요한 것은 단순 observability/logging이 아니라 trajectory data management.
- 특히 trajectory → derived memory/judgment → 다시 trajectory로 provenance 추적하는 lineage가 핵심 abstraction.
- Agent가 길어질수록 DB도 단순 CRUD가 아니라 시간적 순서 + graph 관계 + LLM semantic query를 함께 최적화해야 함.
- 즉, “LLM Agent를 위한 DB”가 아니라 “Agent의 실행 history 자체를 데이터 타입으로 보는 DB”가 이 논문의 가장 중요한 관점 전환.